PRAKTEK TEKNOLOGI IOT II

LAPORAN PRAKTEK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum

Mata Kuliah Teknologi IOT II

DISUSUN OLEH :

NAMA : Muhammad Daffa Al Barru

NPM : 25782017

PRODI : TEKNOLOGI REKAYASA INTERNET

KELAS : 3A

DOSEN : Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

: Agus Ambarwari, S.Pd.,M.Kom.

MENTRI PENDIDIKAN DAN KEBUDAYAAN PROVINSI LAMPUNG

POLITEKNIK NEGERI LAMPUNG

TAHUN AJARAN 2025 / 2026

7.5 Praktikum

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Pada tahap ini, dilakukan pembangunan infrastruktur server lokal terpadu yang mencakup *MQTT Broker* (Eclipse Mosquitto) dan *Visual Rule Engine* (Node-RED) secara otomatis menggunakan kontainerisasi Docker**.**

Langkah Kerja & Implementasi:

1. Membuat Struktur Direktori Kerja: Membuat folder utama bernama iot-server di dalam direktori lokal komputer, beserta sub-direktori iot-server/mosquitto/config/.
2. Konfigurasi Mosquitto (mosquitto.conf): Berkas konfigurasi dibuat di dalam folder mosquitto/config/ dengan parameter utama sebagai berikut:

mosquitto.conf - Notepad

File Edit Format View Help

A new version of Notepad is available. Launch

```
# ============================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ============================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

3. Penyusunan Berkas Orkestrasi (docker-compose.yml): Berkas orkestrasi multi-kontainer ditempatkan di direktori utama iot-server/:

compose.yaml - Notepad

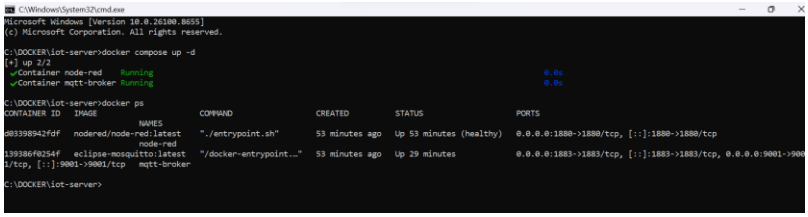
File Edit Format View Help

A new version of Notepad is available. Launch

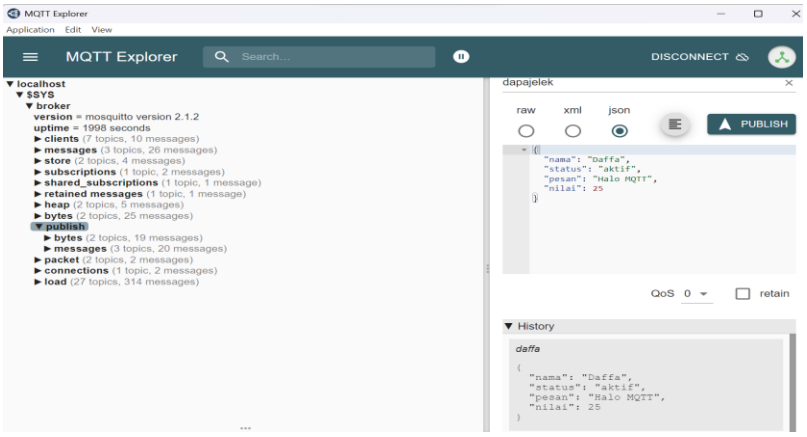
```
services:
  # ============================================================
  # 1. Layanan Broker MQTT: Eclipse Mosquitto
  # ============================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port MQTT murni untuk mikrokontroler
      - "9001:9001"  # Port WebSockets untuk dashboard web
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

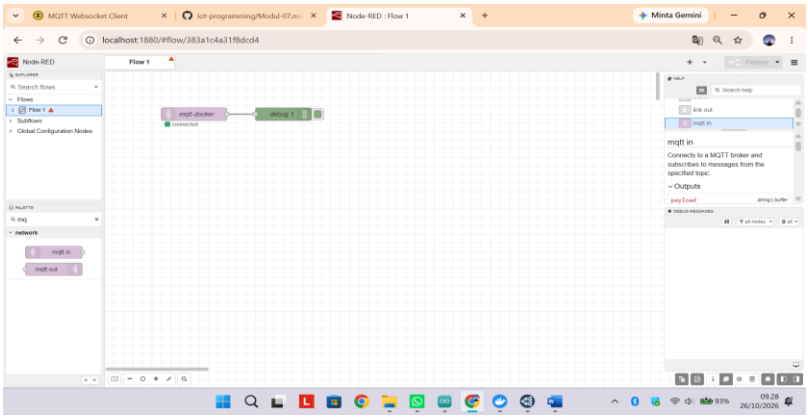
  # ============================================================
  # 2. Layanan Visual Rule Engine: Node-RED
  # ============================================================
  nodered:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"  # Port antarmuka web builder Node-RED
    environment:
      - TZ=Asia/Jakarta
    volumes:
      - ./nodered:/data
    depends_on:
      - mqtt
```

4. **Eksekusi Kontainer:** Melalui terminal PowerShell, direktori diarahkan ke folder iot-server, lalu perintah orkestrasi dijalankan:
5. **Verifikasi Proses:** Pemeriksaan status kontainer menggunakan perintah docker ps memastikan bahwa kontainer mqtt-broker dan node-red telah berstatus aktif (*Up*).



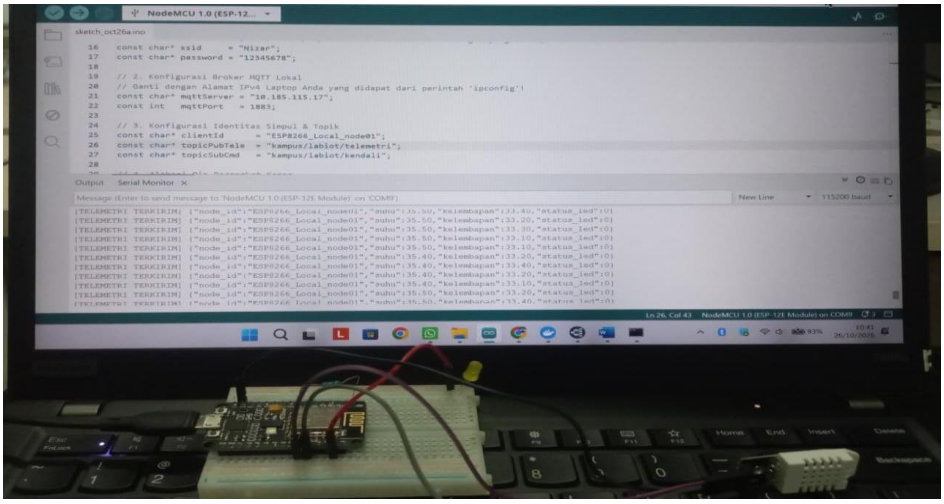
6. **Pengujian Akses Web:** Buka peramban web pada laptop, lalu ketikkan alamat URL: http://localhost:1880.

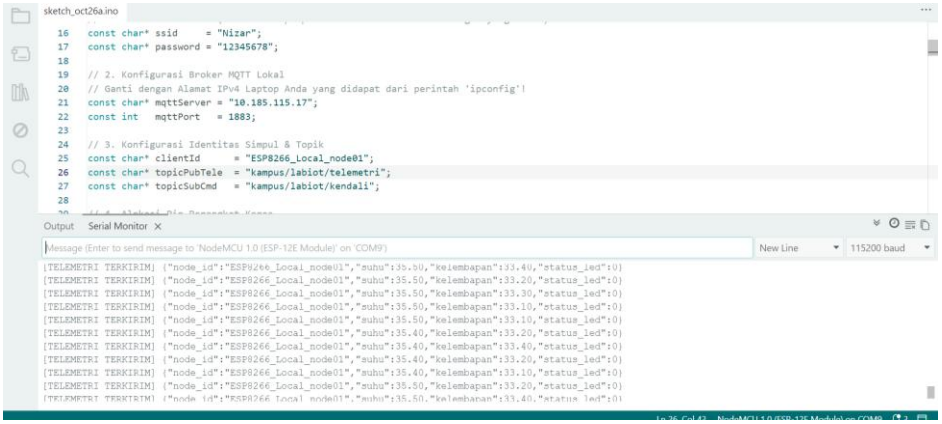




7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

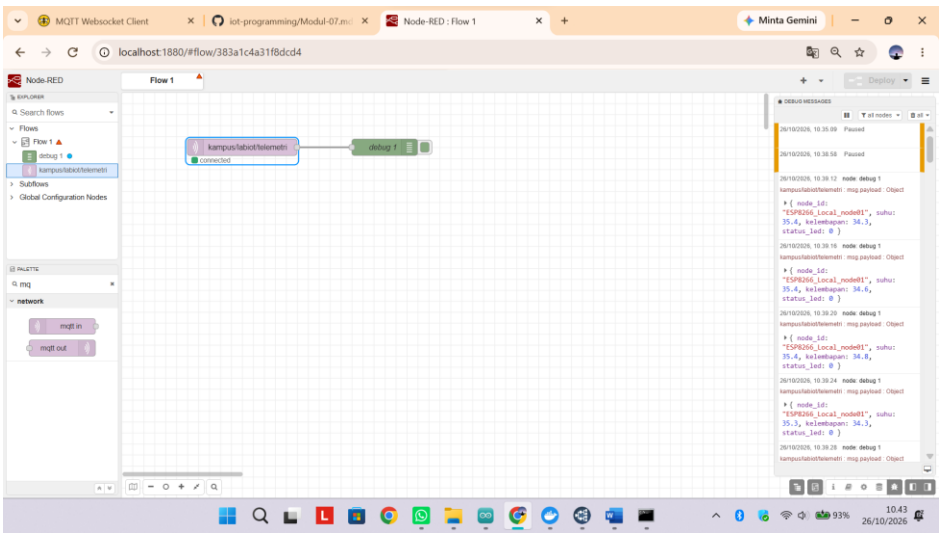
Setelah server lokal berjalan di laptop, langkah berikutnya adalah mengarahkan NodeMCU ESP8266 dari broker publik ke alamat IP lokal laptop Anda dalam jaringan Wi-Fi lokal.





7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

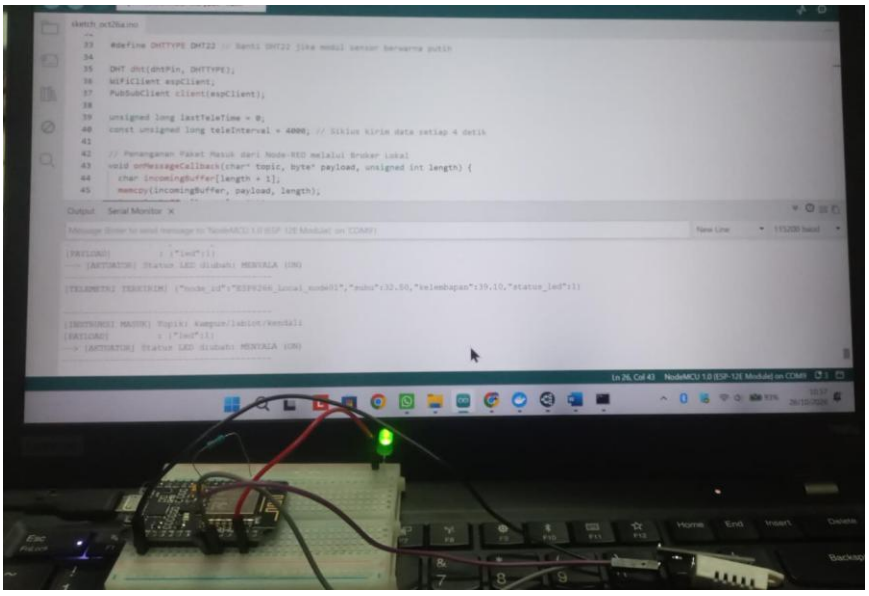
Pada praktikum ini, kita akan menangkap dan membedah aliran data telemetri yang dikirimkan oleh NodeMCU menggunakan antarmuka kanvas Node-RED.

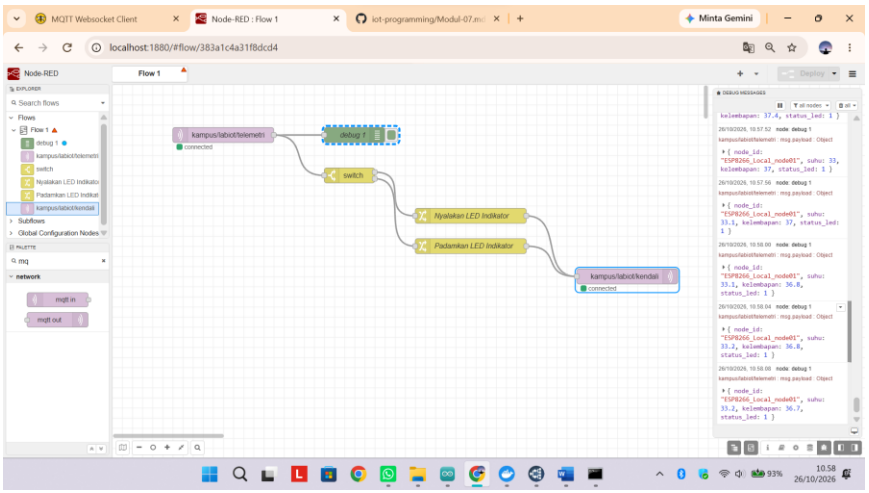


7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

- **JIKA** nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis **31.0∘C**: Node-RED secara otomatis memublikasikan paket JSON {"led": 1} untuk menyalakan LED indikator peringatan bahaya di pin D1.
- **JIKA** suhu berada dalam batas normal (≤31.0∘C): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.





7.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan eksperimen teknis untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)? Di dalam arsitektur Docker, setiap kontainer berjalan di dalam ruang *namespace* jaringan yang terisolasi. Jika aplikasi di dalam kontainer Node-RED mencoba mengakses localhost (127.0.0.1), sistem akan merujuk ke antarmuka *loopback* milik kontainer Node-RED itu sendiri, bukan ke laptop (*host*) ataupun kontainer Mosquitto. Karena layanan Mosquitto tidak berjalan di dalam kontainer Node-RED, koneksi akan ditolak secara otomatis (*Connection Refused*).

  ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya! Ketika beberapa kontainer pada berkas docker-compose.yml dihubungkan ke dalam satu jaringan *Docker Bridge* yang sama, Docker secara otomatis mengaktifkan layanan *Embedded DNS Server*. Server DNS internal ini bertugas menerjemahkan nama layanan (*service name*), seperti mqtt (untuk kontainer Mosquitto), langsung menjadi alamat IP internal kontainer yang bersangkutan. Oleh karena itu, Node-RED dapat berkomunikasi dengan lancar hanya dengan menggunakan nama mqtt sebagai alamat hos.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut? Kedua perangkat berada di dua *subnet* jaringan fisik yang berbeda. NodeMCU terhubung ke jaringan *hotspot* ponsel pada segmen 192.168.43.x, sementara laptop berada pada segmen jaringan lokal 192.168.1.x. Perangkat yang berada pada subnet berbeda tidak dapat saling mengirimkan paket data secara langsung tanpa adanya *routing* atau *NAT/port forwarding* khusus pada *gateway*, sehingga paket MQTT dari NodeMCU tidak akan pernah mencapai IP broker di laptop.

ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama? Mahasiswa harus memastikan bahwa laptop & NodeMCU terhubung ke sumber jaringan Wi-Fi yang sama persis. Cara paling praktis adalah dengan menggunakan satu perangkat *Mobile Hotspot* yang sama, sehingga kedua perangkat mendapatkan alokasi alamat IP dari satu segmen *subnet* yang selaras.

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
  Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0∘C (misal: 31.05∘C→30.95∘C→31.02∘C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem. Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:
  - LED baru menyala jika suhu >31.5∘C (msg.payload = {"led": 1}).
  - LED baru padam jika suhu mendingin hingga <30.0∘C (msg.payload = {"led": 0}).
    Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

```
var suhu = parseFloat(msg.payload.suhu);
var statusLed = context.get('statusLed') || 0;
if (!isNaN(suhu)) {
  if (suhu > 31.5) {
    statusLed = 1; // Menyalakan LED jika suhu melewati batas atas
  } else if (suhu < 30.0) {
    statusLed = 0; // Memadamkan LED jika suhu turun di bawah batas bawah
  }
  context.set('statusLed', statusLed);
    msg.payload = { "led": statusLed };
  return msg;
}
return null;
```

- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml? Seluruh alur kerja (*flows*), konfigurasi dasbor, serta data di dalam direktori internal /data pada Node-RED akan hilang total (*reset* ke pengaturan awal). Hal ini terjadi karena kontainer bersifat sementara (*ephemeral*); tanpa adanya pemetaan volume (*Volume Mount*) ke penyimpanan pada komputer *host*, seluruh data di dalam kontainer akan terhapus saat kontainer dihentikan atau dihapus (docker compose down).

  ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

  Bind Mount: Mengarahkan langsung ke folder spesifik di komputer host (misal ./nodered:/data).

  Named Volume: Penyimpanan dikelola secara otomatis di dalam area khusus oleh Docker tanpa perlu menentukan folder host secara manual.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Dalam dunia industri nyata, membiarkan MQTT Broker beroperasi tanpa kata sandi (allow_anonymous true) adalah pelanggaran fatal terhadap standar keamanan siber (*Cybersecurity Vulnerability*). Pada tugas ini, kembangkan ekosistem server IoT lokal Anda menjadi sistem yang terproteksi penuh menggunakan sistem autentikasi kredensial (*Username & Password*).

Instruksi Pengerjaan Langkah demi Langkah:

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

```
C:\DOCKER\iot-server>docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/

C:\DOCKER\iot-server>
```

2. Memperbarui Konfigurasi Kebijakan Mosquitto:

*mosquitto.conf - Notepad

File Edit Format View Help

A new version of Notepad is available. Launch

```
# ================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous false
password_file /mosquitto/config/password.txt

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

3. Memuat Ulang Layanan Broker:

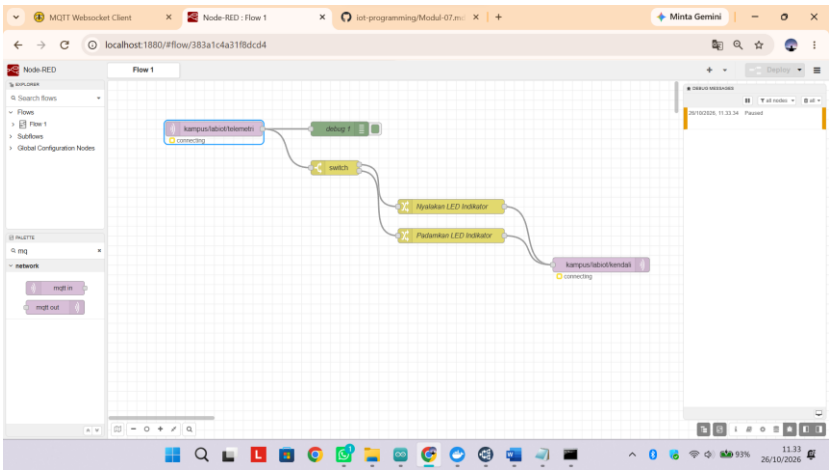
C:\Windows\System32\cmd.exe

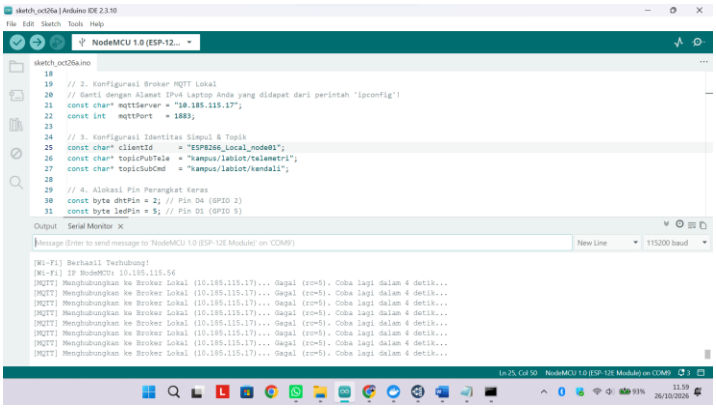
```
Microsoft Windows [Version 10.0.26100.8655]
(c) Microsoft Corporation. All rights reserved.

C:\DOCKER\iot-server>docker restart mqtt-broker
mqtt-broker

C:\DOCKER\iot-server>_
```

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):





5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

